

Web Services Assignment 1

Documentation

FastAPI Python code

```
from fastapi import FastAPI, HTTPException
from pymongo.mongo_client import MongoClient
from pydantic import BaseModel
import requests
from prometheus_fastapi_instrumentator import Instrumentator

import os
from dotenv import load_dotenv

load_dotenv()

app = FastAPI()

# API monitoring
Instrumentator().instrument(app).expose(app)

# DB connection to Mongo Atlas
uri = os.getenv("MONGO_URI")
client = MongoClient(uri)
db = client["inventory_db"]
collection = db["products"]

# Pydantic model for adding new product
class Product(BaseModel):
    ProductID: int
    Name: str
    UnitPrice: float
    StockQuantity: int
    Description: str

# Helper to strip MongoDB's _id field from results
def clean(product):
    product["_id"] = str(product["_id"])
    return product

# Endpoints
@app.get("/getSingleProduct")
def get_single_product(id: int):
    product = collection.find_one({"ProductID": id})
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")
    return clean(product)

@app.get("/getAll")
def get_all():
    products = list(collection.find())
    return [clean(p) for p in products]
```

```

@app.post("/addNew")
def add_new(product: Product):
    existing = collection.find_one({"ProductID": product.ProductID})
    if existing:
        raise HTTPException(status_code=400, detail="ProductID already exists")
    collection.insert_one(product.dict())
    return {"message": "Product added successfully"}

@app.delete("/deleteOne")
def delete_one(id: int):
    result = collection.delete_one({"ProductID": id})
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Product not found")
    return {"message": f"Product {id} deleted successfully"}

@app.get("/startsWith")
def starts_with(letter: str):
    if len(letter) != 1 or not letter.isalpha():
        raise HTTPException(status_code=400, detail="Please provide a single letter")
    products = list(collection.find({"Name": {"$regex": f"^{letter}", "$options": "i"}}))
    return [clean(p) for p in products]

@app.get("/paginate")
def paginate(start_id: int, end_id: int):
    products = list(collection.find(
        {"ProductID": {"$gte": start_id, "$lte": end_id}}
    ).limit(10))
    return [clean(p) for p in products]

@app.get("/convert")
def convert(id: int):
    product = collection.find_one({"ProductID": id})
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")

    # Fetch live USD to EUR exchange rate
    response = requests.get("https://api.exchangerate-api.com/v4/latest/USD")
    if response.status_code != 200:
        raise HTTPException(status_code=500, detail="Could not fetch exchange rate")

    rate = response.json()["rates"]["EUR"]
    price_in_euros = round(product["UnitPrice"] * rate, 2)

    return {
        "ProductID": product["ProductID"],
        "Name": product["Name"],
        "OriginalPrice_USD": product["UnitPrice"],
        "ConvertedPrice_EUR": price_in_euros,
        "ExchangeRate": rate
    }

```

Unit test code

Postman screenshots

Web Assignment Tests > Get Single Product

GET http://localhost:8000/getSingleProduct?id=1001

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request
Post-response ●

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("Response is JSON", function () {
5   pm.response.to.be.json;
6 });
```

Body Cookies Headers 4 Test Results 2/2 200 OK • 215 ms

{ } JSON Preview Visualize

```
1 {
2   "_id": "69bbe058acb01f94557e3ba8",
3   "ProductID": 1001,
4   "Name": "NVIDIA RTX 4090",
5   "UnitPrice": 1599.99,
6   "StockQuantity": 12,
7   "Description": "High-end GPU with 24GB VRAM for 4K gaming."
8 }
```

Web Assignment Tests > Get All

GET http://localhost:8000/getAll

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request
Post-response ●

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("Response is JSON", function () {
5   pm.response.to.be.json;
6 });
```

Body Cookies Headers 4 Test Results 2/2 200 OK • 77 ms

{ } JSON Preview Visualize

```
1 [
2   {
3     "_id": "69bbe058acb01f94557e3ba8",
4     "ProductID": 1001,
5     "Name": "NVIDIA RTX 4090",
6     "UnitPrice": 1599.99,
7     "StockQuantity": 12,
8     "Description": "High-end GPU with 24GB VRAM for 4K gaming."
9   },
10  {
11    "_id": "69bbe058acb01f94557e3ba9",
12    "ProductID": 1002,
13    "Name": "AMD Ryzen 9 7950X",
14    "UnitPrice": 549.0,
15    "StockQuantity": 25,
16    "Description": "16-core 32-thread enthusiast desktop processor."
17  },
18 ]
```

POST http://localhost:8000/addNew

Docs Params Authorization Headers 8 Body Scripts Settings

Pre-request
Post-response ●

```
1 pm.test("Status code is 200", function () {  
2 |   pm.response.to.have.status(200);  
3 | });  
4 pm.test("Response is JSON", function () {  
5 |   pm.response.to.be.json;  
6 | });
```

Body Cookies Headers 4 Test Results 2/2

200 OK • 56 ms

{ } JSON Preview Visualize

```
1 {  
2 |   "message": "Product added successfully"  
3 | }
```

DELETE http://localhost:8000/deleteOne?id=9999

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request
Post-response ●

```
1 pm.test("Status code is 200", function () {  
2 |   pm.response.to.have.status(200);  
3 | });  
4 pm.test("Response is JSON", function () {  
5 |   pm.response.to.be.json;  
6 | });
```

Body Cookies Headers 4 Test Results 2/2

200 OK • 26 ms

{ } JSON Preview Visualize

```
1 {  
2 |   "message": "Product 9999 deleted successfully"  
3 | }
```

GET
http://localhost:8000/startsWith?letter=E

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request
Post-response

```

1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("Response is JSON", function () {
5   pm.response.to.be.json;
6 });

```

Body Cookies Headers 4 Test Results 2/2 200 OK • 24 ms

JSON Preview Visualize

```

1 [
2   {
3     "_id": "69bbe058acb01f94557e3bad",
4     "ProductID": 1006,
5     "Name": "EVGA SuperNova 850 G6",
6     "UnitPrice": 139.5,
7     "StockQuantity": 30,
8     "Description": "850W 80 Plus Gold Fully Modular Power Supply."
9   },
10  {
11    "_id": "69bbe058acb01f94557e3bf8",
12    "ProductID": 1081,
13    "Name": "EVGA RTX 3060 Ti",
14    "UnitPrice": 399.0,
15    "StockQuantity": 12,
16    "Description": "Excellent 1440p performer (Last Gen)."
17  },

```

GET
http://localhost:8000/paginate?start_id=1003&end_id=1008

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request
Post-response

```

1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("Response is JSON", function () {
5   pm.response.to.be.json;
6 });

```

Body Cookies Headers 4 Test Results 2/2 200 OK • 22 ms

JSON Preview Visualize

```

1 [
2   {
3     "_id": "69bbe058acb01f94557e3baa",
4     "ProductID": 1003,
5     "Name": "Samsung 990 Pro 2TB",
6     "UnitPrice": 179.99,
7     "StockQuantity": 40,
8     "Description": "PCIe Gen4 NVMe M.2 SSD with 7450MB/s reads."
9   },
10  {
11    "_id": "69bbe058acb01f94557e3bab",
12    "ProductID": 1004,
13    "Name": "Corsair Vengeance 32GB DDR5",
14    "UnitPrice": 115.0,
15    "StockQuantity": 60,
16    "Description": "6000MHz CL36 Dual Channel Memory Kit."
17  },

```

GET

http://localhost:8000/convert?id=1002

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Pre-request

Post-response

```

1  pm.test("Status code is 200", function () {
2    |    pm.response.to.have.status(200);
3  });
4  pm.test("Response is JSON", function () {
5    |    pm.response.to.be.json;
6  });

```

Body

Cookies

Headers 4

Test Results 2/2

200 OK • 613 ms

{ } JSON

Preview

Visualize

```

1  {
2    |    "ProductID": 1002,
3    |    "Name": "AMD Ryzen 9 7950X",
4    |    "OriginalPrice_USD": 549.0,
5    |    "ConvertedPrice_EUR": 475.43,
6    |    "ExchangeRate": 0.866
7  }

```

Newman screenshot

→ Convert

GET <http://localhost:8000/convert?id=1002> [200 OK, 246B, 641ms]

- ✓ Status code is 200
- ✓ Response is JSON

	executed	failed
iterations	1	0
requests	7	0
test-scripts	7	0
prerequest-scripts	0	0
assertions	14	0
total run duration: 1218ms		
total data received: 38.11kB (approx)		
average response time: 151ms [min: 24ms, max: 641ms, s.d.: 207ms]		

Docker config

FROM ubuntu:22.04

RUN apt-get update && apt-get install -y \
python3 \
python3-pip \
&& rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip3 install -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

Endpoints API is running

FastAPI 0.1.0 OAS 3.1

/openapi.json

default



GET	/metrics	Metrics	⌵
GET	/getSingleProduct	Get Single Product	⌵
GET	/getAll	Get All	⌵
POST	/addNew	Add New	⌵
DELETE	/deleteOne	Delete One	⌵
GET	/startsWith	Starts With	⌵
GET	/paginate	Paginate	⌵
GET	/convert	Convert	⌵

Jenkins pipeline

Jenkinsfile

```
pipeline {
    agent any

    stages {

        stage('Clone Repo') {
            steps {
                echo 'Code pulled from GitHub successfully'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t web-assignment ./web-assignment'
            }
        }

        stage('Run Container') {
            steps {
                withCredentials([string(credentialsId: 'MONGO_URI', variable: 'MONGO_URI')]) {
                    sh 'docker rm -f web-assignment-container || true'
                    sh 'docker run -d -p 8000:8000 --name web-assignment-container -e MONGO_URI=$MONGO_URI web-assignment'
                    sh 'sleep 5'
                }
            }
        }

        stage('Run Tests') {
            steps {
                sh 'cd web-assignment && newman run tests/postman_collection.json'
            }
        }

        stage('Generate README') {
            steps {
                sh '''
cat > README.txt << EOF
Web Services Assignment 1 - API Endpoints
=====

GET /getSingleProduct?id=1          - Returns a single product by ID
GET /getAll                        - Returns all products
POST /addNew                       - Adds a new product (JSON body required)
DELETE /deleteOne?id=1             - Deletes a product by ID
GET /startsWith?letter=s           - Returns all products starting with a letter
GET /paginate?start_id=1&end_id=50 - Returns up to 10 products in ID range
GET /convert?id=1                  - Returns product price converted to EUR

FastAPI Interactive Docs: http://localhost:8000/docs
Full FastAPI documentation: https://fastapi.tiangolo.com/
EOF
'''
            }
        }
    }
}
```

```

stage('Package') {
  steps {
    sh 'zip -r complete-$(date +%Y%m%d-%H%M%S).zip web-assignment/ README.txt'
    archiveArtifacts artifacts: 'complete-*.zip, README.txt', fingerprint: true
  }
}

stage('Cleanup') {
  steps {
    sh 'docker stop web-assignment-container'
    sh 'docker rm web-assignment-container'
  }
}
}

```

Jenkins successful build



Contents of final zip created by Jenkins

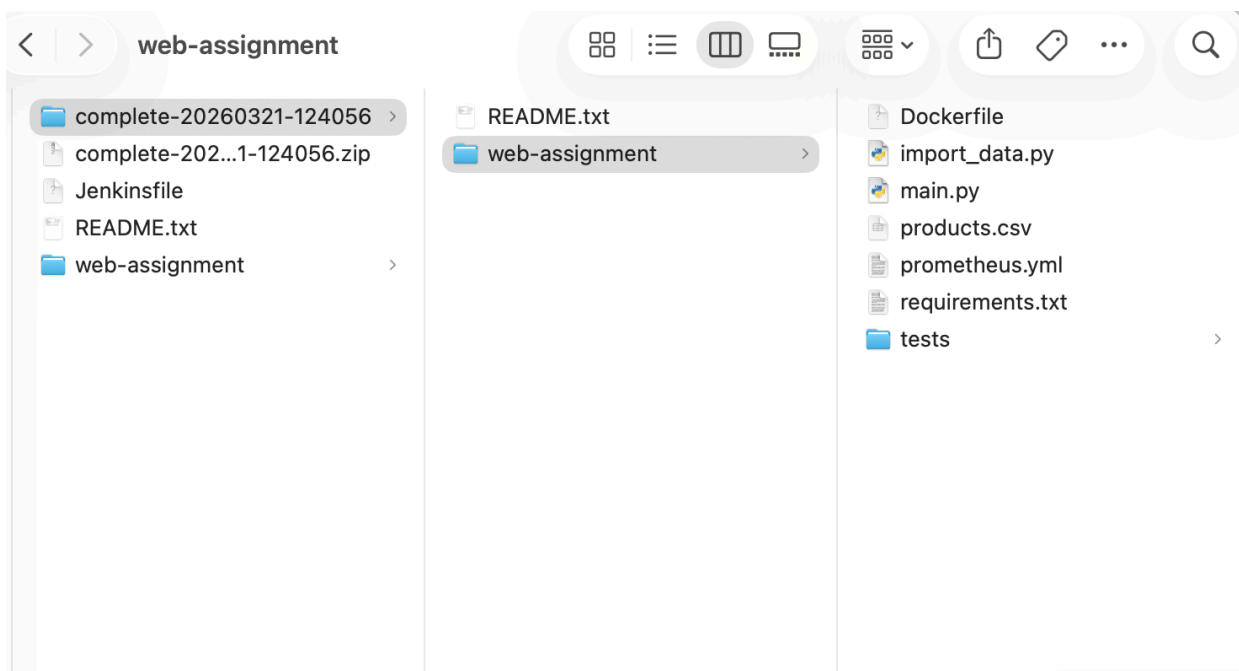


web-assignment-pipeline



Last Successful Artifacts

 complete-20260320-200608.zip	13.35 KiB	 view
 complete-20260320-200622.zip	13.35 KiB	 view
 complete-20260320-200651.zip	13.35 KiB	 view
 complete-20260320-201541.zip	13.35 KiB	 view
 README.txt	708 B	 view



complete-datetime.zip file contains README.txt, project code

API Monitoring (Grafana dashboard output)



/convert endpoint takes longer due to converting currency through an API.